TAKE-HOME ASSIGNMENT • SDE2

Real-Time Charting Application

Live candlestick charting on the KlineCharts library

Build a real-time charting application on top of the open-source KlineCharts library. The application should render live candlestick charts, support multiple time intervals, update with streaming data, allow multi-layout chart views and workspace saving, and run on a scalable backend that delivers real-time market data to a large number of concurrent users.

This assignment reflects the kind of work you would do with us: extending an existing codebase, making sound architecture decisions, and building systems that hold up under real load. We are as interested in how you reason about trade-offs as in the finished product.

01 Project Setup

- Clone the **KlineCharts** library from its official GitHub repository.
- Set up the development environment and install the required dependencies.
- Build your UI **inside the existing KlineCharts codebase**. Do not scaffold a separate project.
- **Do not add any new external libraries to `package.json`.** Work with what the library already ships. Backend dependencies in a separate service are acceptable, as described in section 03.

02 Frontend Requirements

- A **responsive UI** that displays a candlestick chart using KlineCharts.
- **Interval switching** between at least 1 minute, 5 minutes, 15 minutes, 1 hour, and 1 day.
- Charts must **update live** as new data arrives from the backend.
- **Multi-layout** chart views, for example a 1x1 view and a 2x2 grid, so a user can watch multiple charts or intervals at once.
- **Workspace saving.** A user's layout, selected symbols, and intervals should persist and be restorable.

UI reference and color theme

Follow the Trading Technologies chart overview for visual direction:

https://library.tradingtechnologies.com/trade/analytics/charts/description-charts/chart-overview/

> We care more about clean architecture and a working real-time pipeline than about pixel-perfect UI. Match the theme reasonably and do not over-invest in polish at the expense of the core system.

03 Backend Requirements

- Set up a backend in your preferred tech stack. It should be **highly scalable** and capable of handling **real-time data updates**.
- It must support **1,000 concurrent users** receiving real-time updates. Write a **load-testing script**, run it, and attach the **benchmark results and a screenshot** as proof.
- **Data pipeline.** Generate mock data only for the lowest timeframe, which is **1 minute**. Build a processing pipeline that derives all higher timeframes (5 minutes, 15 minutes, 1 hour, 1 day) **from the 1-minute data only**.
- **Aggregation correctness.** Higher-interval candles must roll up correctly: open taken from the first sub-candle, close from the last, high and low as the extremes across the window, and volume summed.

04 Design Write-Up (required)

Include a short document (Markdown is fine) that covers:

- **System architecture.** How the frontend, backend, and data pipeline fit together, with a diagram.
- **Scalability strategy.** How your design supports 1,000 concurrent connections, and where it would break next and why.
- **Real-time transport.** Your choice (WebSocket, Server-Sent Events, or other) and the reasoning behind it.
- **Trade-offs and assumptions.** Anything ambiguous in the brief that you resolved by making a decision.

> If something in this brief is unclear, treat it as a design decision, document your assumption, and move on. That is part of what we are assessing.

05 Deliverables

- A **GitHub repository** (public, or with access granted to us) containing all code.
- A clear **README** with setup and run instructions for both the frontend and the backend.
- The **load-test script** together with its **benchmark output and screenshot**.
- The **design write-up** described in section 04.

06 Evaluation Criteria

AREA	WHAT WE LOOK FOR
Architecture and code quality	Clean structure, separation of concerns, readability, and a sensible extension of the KlineCharts codebase.
Real-time correctness	Live updates work reliably, and timeframe aggregation is correct and derived only from 1-minute data.
Scalability and evidence	The backend handles the concurrency target, and the load test is real with a benchmark that backs the claim.
Frontend functionality	Interval switching, multi-layout, and workspace saving all work and are responsive.
Design write-up and trade-offs	Clear reasoning, honesty about limitations, and well-judged assumptions.

07 Timeline and Submission

TIME TO COMPLETE

3 days

SUBMIT BY

Monday, end of day, 11:00 PM

How to submit. Reply to this email with your repository link and the design write-up.

Questions. If you hit a genuine blocker, reach out. We would rather you ask than stay stuck.